

- b. Derive the error term in Theorem 3.9. [Hint: Use the same method as in the Lagrange error derivation, Theorem 3.3, defining

$$g(t) = f(t) - H_{2n+1}(t) - \frac{(t-x_0)^2 \cdots (t-x_n)^2}{(x-x_0)^2 \cdots (x-x_n)^2} [f(x) - H_{2n+1}(x)]$$

and using the fact that  $g'(t)$  has  $(2n+2)$  distinct zeros in  $[a, b]$ .]

9. Let  $z_0 = x_0$ ,  $z_1 = x_0$ ,  $z_2 = x_1$ , and  $z_3 = x_1$ . Form the following divided-difference table.

|             |                   |                         |                    |                         |
|-------------|-------------------|-------------------------|--------------------|-------------------------|
| $z_0 = x_0$ | $f[z_0] = f(x_0)$ |                         |                    |                         |
|             |                   | $f[z_0, z_1] = f'(x_0)$ |                    |                         |
| $z_1 = x_0$ | $f[z_1] = f(x_0)$ |                         | $f[z_0, z_1, z_2]$ |                         |
|             |                   | $f[z_1, z_2]$           |                    | $f[z_0, z_1, z_2, z_3]$ |
| $z_2 = x_1$ | $f[z_2] = f(x_1)$ |                         | $f[z_1, z_2, z_3]$ |                         |
|             |                   | $f[z_2, z_3] = f'(x_1)$ |                    |                         |
| $z_3 = x_1$ | $f[z_3] = f(x_1)$ |                         |                    |                         |

Show that the cubic Hermite polynomial  $H_3(x)$  can also be written as  $f[z_0] + f[z_0, z_1](x - x_0) + f[z_0, z_1, z_2](x - x_0)^2 + f[z_0, z_1, z_2, z_3](x - x_0)^2(x - x_1)$ .

## 3.4 Cubic Spline Interpolation\*

The previous sections concerned the approximation of arbitrary functions on closed intervals by the use of polynomials. However, the oscillatory nature of high-degree polynomials and the property that a fluctuation over a small portion of the interval can induce large fluctuations over the entire range restricts their use. We will see a good example of this in Figure 3.12 at the end of this section.

An alternative approach is to divide the interval into a collection of subintervals and construct a (generally) different approximating polynomial on each subinterval. Approximation by functions of this type is called **piecewise-polynomial approximation**.

The simplest piecewise-polynomial approximation is piecewise-linear interpolation, which consists of joining a set of data points

$$\{(x_0, f(x_0)), (x_1, f(x_1)), \dots, (x_n, f(x_n))\}$$

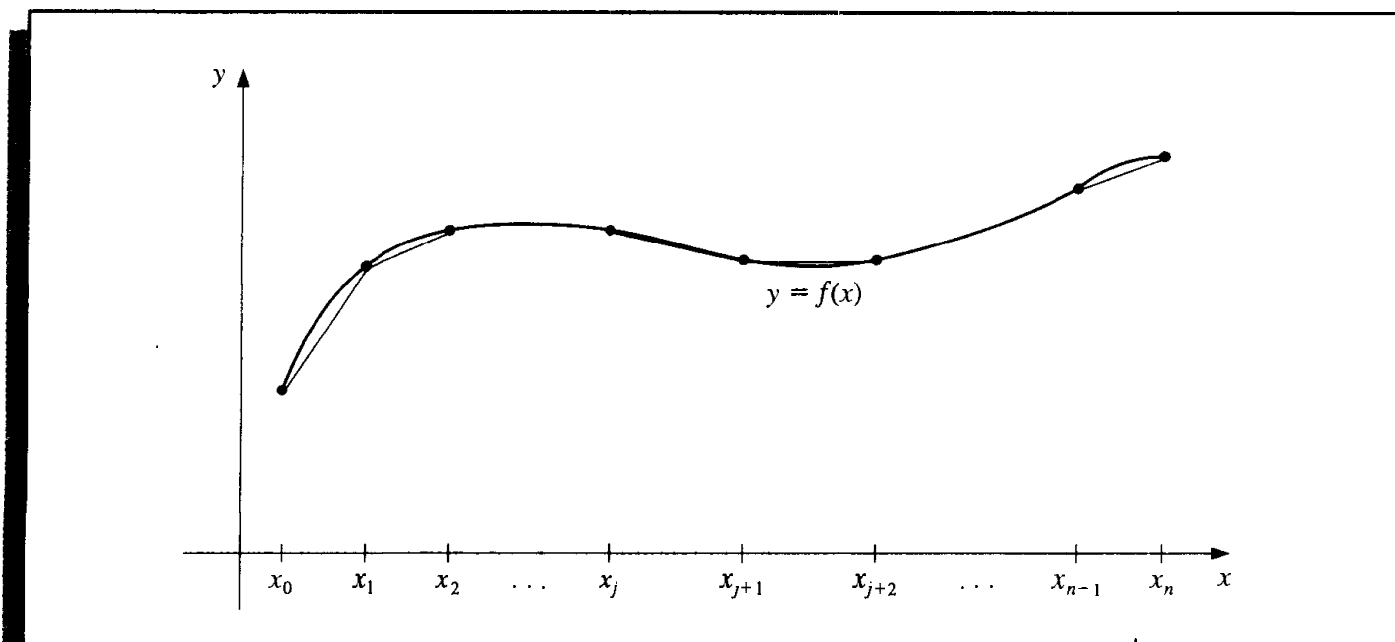
by a series of straight lines, as shown in Figure 3.7 on page 142.

A disadvantage of linear function approximation is that there is likely no differentiability at the endpoints of the subintervals, which, in a geometrical context, means that the interpolating function is not “smooth.” Often it is clear from physical conditions that smoothness is required, so the approximating function must be continuously differentiable.

An alternative procedure is to use a piecewise polynomial of Hermite type. For example, if the values of  $f$  and of  $f'$  are known at each of the points  $x_0 < x_1 < \cdots < x_n$ , a cubic Hermite polynomial can be used on each of the subintervals  $[x_0, x_1]$ ,  $[x_1, x_2]$ ,  $\dots$ ,  $[x_{n-1}, x_n]$  to obtain a function that has a continuous derivative on the interval  $[x_0, x_n]$ . To

\*The proofs of the theorems in this section rely on results in Chapter 6.

Figure 3.7



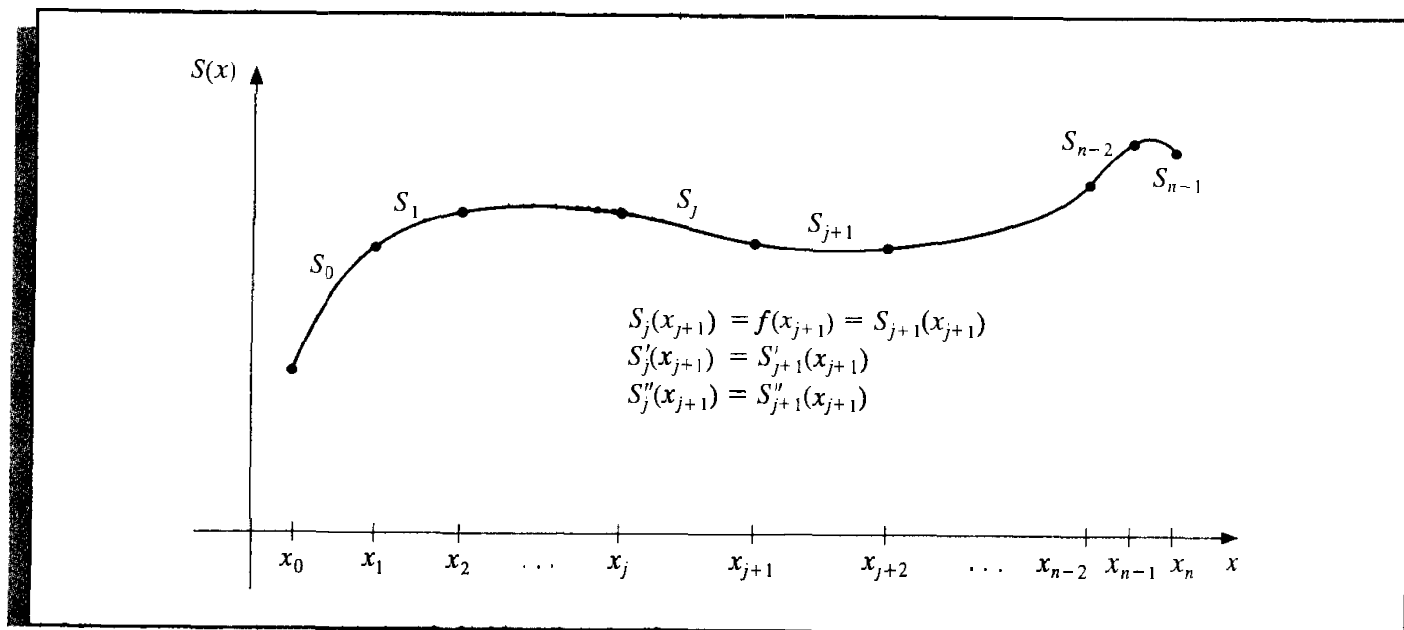
determine the appropriate Hermite cubic polynomial on a given interval is simply a matter of computing  $H_3(x)$  for that interval. Since the Lagrange interpolating polynomials needed to determine  $H_3$  are of first degree, this can be accomplished without great difficulty. However, to use Hermite piecewise polynomials for general interpolation, we need to know the derivative of the function being approximated, which is frequently unavailable.

The remainder of this section considers approximation using piecewise polynomials that require no derivative information, except perhaps at the endpoints of the interval on which the function is being approximated.

The simplest type of differentiable piecewise-polynomial function on an entire interval  $[x_0, x_n]$  is the function obtained by fitting one quadratic polynomial between each successive pair of nodes. This is done by constructing one quadratic on  $[x_0, x_1]$  agreeing with the function at  $x_0$  and  $x_1$ , another quadratic on  $[x_1, x_2]$  agreeing with the function at  $x_1$  and  $x_2$ , and so on. Since a general quadratic polynomial has three arbitrary constants—the constant term, the coefficient of  $x$ , and the coefficient of  $x^2$ —and only two conditions are required to fit the data at the endpoints of each subinterval, flexibility exists that permits the quadratic to be chosen so that the interpolant has a continuous derivative on  $[x_0, x_n]$ . The difficulty arises when there is a need to specify conditions about the derivative of the interpolant at the endpoints  $x_0$  and  $x_n$ . There is not a sufficient number of constants to ensure that the conditions will be satisfied. (See Exercise 22.)

The most common piecewise-polynomial approximation uses cubic polynomials between each successive pair of nodes and is called **cubic spline interpolation**. A general cubic polynomial involves four constants, so there is sufficient flexibility in the cubic spline procedure to ensure that the interpolant is not only continuously differentiable on the interval, but also has a continuous second derivative. The construction of the cubic spline does not, however, assume that the derivatives of the interpolant agree with those of the function it is approximating, even at the nodes. (See Figure 3.8.)

Figure 3.8



**Definition 3.10** Given a function  $f$  defined on  $[a, b]$  and a set of nodes  $a = x_0 < x_1 < \dots < x_n = b$ , a **cubic spline interpolant**  $S$  for  $f$  is a function that satisfies the following conditions:

- a.  $S(x)$  is a cubic polynomial, denoted  $S_j(x)$ , on the subinterval  $[x_j, x_{j+1}]$  for each  $j = 0, 1, \dots, n-1$ ;
- b.  $S(x_j) = f(x_j)$  for each  $j = 0, 1, \dots, n$ ;
- c.  $S_{j+1}(x_{j+1}) = S_j(x_{j+1})$  for each  $j = 0, 1, \dots, n-2$ ;
- d.  $S'_{j+1}(x_{j+1}) = S'_j(x_{j+1})$  for each  $j = 0, 1, \dots, n-2$ ;
- e.  $S''_{j+1}(x_{j+1}) = S''_j(x_{j+1})$  for each  $j = 0, 1, \dots, n-2$ ;
- f. One of the following sets of boundary conditions is satisfied:
  - (i)  $S''(x_0) = S''(x_n) = 0$  (**free or natural boundary**);
  - (ii)  $S'(x_0) = f'(x_0)$  and  $S'(x_n) = f'(x_n)$  (**clamped boundary**).

Although cubic splines are defined with other boundary conditions, the conditions given in (f) are sufficient for our purposes. When the free boundary conditions occur, the spline is called a **natural spline**, and its graph approximates the shape that a long flexible rod would assume if forced to go through the data points  $\{(x_0, f(x_0)), (x_1, f(x_1)), \dots, (x_n, f(x_n))\}$ .

In general, clamped boundary conditions lead to more accurate approximations since they include more information about the function. However, for this type of boundary condition to hold, it is necessary to have either the values of the derivative at the endpoints or an accurate approximation to those values.

To construct the cubic spline interpolant for a given function  $f$ , the conditions in the definition are applied to the cubic polynomials

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3,$$

for each  $j = 0, 1, \dots, n-1$ .

Since

$$S_j(x_j) = a_j = f(x_j),$$

condition (c) can be applied to obtain

$$a_{j+1} = S_{j+1}(x_{j+1}) = S_j(x_{j+1}) = a_j + b_j(x_{j+1} - x_j) + c_j(x_{j+1} - x_j)^2 + d_j(x_{j+1} - x_j)^3,$$

for each  $j = 0, 1, \dots, n-2$ .

Since the terms  $x_{j+1} - x_j$  are used repeatedly in this development, it is convenient to introduce the simpler notation

$$h_j = x_{j+1} - x_j,$$

for each  $j = 0, 1, \dots, n-1$ . If we also define  $a_n = f(x_n)$ , then the equation

$$a_{j+1} = a_j + b_j h_j + c_j h_j^2 + d_j h_j^3 \quad (3.15)$$

holds for each  $j = 0, 1, \dots, n-1$ .

In a similar manner, define  $b_n = S'(x_n)$  and observe that

$$S'_j(x) = b_j + 2c_j(x - x_j) + 3d_j(x - x_j)^2$$

implies  $S'_j(x_j) = b_j$ , for each  $j = 0, 1, \dots, n-1$ . Applying condition (d) gives

$$b_{j+1} = b_j + 2c_j h_j + 3d_j h_j^2, \quad (3.16)$$

for each  $j = 0, 1, \dots, n-1$ .

Another relationship between the coefficients of  $S_j$  is obtained by defining  $c_n = S''(x_n)/2$  and applying condition (e). Then, for each  $j = 0, 1, \dots, n-1$ ,

$$c_{j+1} = c_j + 3d_j h_j. \quad (3.17)$$

Solving for  $d_j$  in Eq. (3.17) and substituting this value into Eqs. (3.15) and (3.16) gives, for each  $j = 0, 1, \dots, n-1$ , the new equations

$$a_{j+1} = a_j + b_j h_j + \frac{h_j^2}{3}(2c_j + c_{j+1}) \quad (3.18)$$

and

$$b_{j+1} = b_j + h_j(c_j + c_{j+1}). \quad (3.19)$$

The final relationship involving the coefficients is obtained by solving the appropriate equation in the form of equation (3.18), first for  $b_j$ ,

$$b_j = \frac{1}{h_j}(a_{j+1} - a_j) - \frac{h_j}{3}(2c_j + c_{j+1}), \quad (3.20)$$

and then, with a reduction of the index, for  $b_{j-1}$ . This gives

$$b_{j-1} = \frac{1}{h_{j-1}}(a_j - a_{j-1}) - \frac{h_{j-1}}{3}(2c_{j-1} + c_j).$$

Substituting these values into the equation derived from Eq. (3.19), with the index reduced by one, gives the linear system of equations

$$h_{j-1}c_{j-1} + 2(h_{j-1} + h_j)c_j + h_jc_{j+1} = \frac{3}{h_j}(a_{j+1} - a_j) - \frac{3}{h_{j-1}}(a_j - a_{j-1}), \quad (3.21)$$

for each  $j = 1, 2, \dots, n-1$ . This system involves only the  $\{c_j\}_{j=0}^n$  as unknowns since the values of  $\{h_j\}_{j=0}^{n-1}$  and  $\{a_j\}_{j=0}^n$  are given, respectively, by the spacing of the nodes  $\{x_j\}_{j=0}^n$  and the values of  $f$  at the nodes.

Note that once the values of  $\{c_j\}_{j=0}^n$  are determined, it is a simple matter to find the remainder of the constants  $\{b_j\}_{j=0}^{n-1}$  from Eq. (3.20) and  $\{d_j\}_{j=0}^{n-1}$  from Eq. (3.17), and to construct the cubic polynomials  $\{S_j(x)\}_{j=0}^{n-1}$ .

The major question that arises in connection with this construction is whether the values of  $\{c_j\}_{j=0}^n$  can be found using the system of equations given in (3.21) and, if so, whether these values are unique. The following theorems indicate that this is the case when either of the boundary conditions given in part (f) of the definition are imposed. The proofs of these theorems require material from linear algebra, which is discussed in Chapter 6.

### Theorem 3.11

If  $f$  is defined at  $a = x_0 < x_1 < \dots < x_n = b$ , then  $f$  has a unique natural spline interpolant  $S$  on the nodes  $x_0, x_1, \dots, x_n$ ; that is, a spline interpolant that satisfies the boundary conditions  $S''(a) = 0$  and  $S''(b) = 0$ . ■

**Proof** The boundary conditions in this case imply that  $c_n = S''(x_n)/2 = 0$  and that

$$0 = S''(x_0) = 2c_0 + 6d_0(x_0 - x_0),$$

so  $c_0 = 0$ .

The two equations  $c_0 = 0$  and  $c_n = 0$  together with the equations in (3.21) produce a linear system described by the vector equation  $A\mathbf{x} = \mathbf{b}$ , where  $A$  is the  $(n+1) \times (n+1)$  matrix

$$A = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 & 0 \\ h_0 & 2(h_0 + h_1) & h_1 & \dots & 0 & 0 \\ 0 & h_1 & 2(h_1 + h_2) & h_2 & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & h_{n-2} & 2(h_{n-2} + h_{n-1}) & h_{n-1} & 0 \\ 0 & \dots & 0 & 0 & 0 & 1 \end{bmatrix},$$

and  $\mathbf{b}$  and  $\mathbf{x}$  are the vectors

$$\mathbf{b} = \begin{bmatrix} 0 \\ \frac{3}{h_1}(a_2 - a_1) - \frac{3}{h_0}(a_1 - a_0) \\ \vdots \\ \frac{3}{h_{n-1}}(a_n - a_{n-1}) - \frac{3}{h_{n-2}}(a_{n-1} - a_{n-2}) \\ 0 \end{bmatrix} \quad \text{and} \quad \mathbf{x} = \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix}.$$

The matrix  $A$  is strictly diagonally dominant, so it satisfies the hypotheses of Theorem 6.19 in Section 6.6. Therefore, the linear system has a unique solution for  $c_0, c_1, \dots, c_n$ .

■ ■ ■

The solution to the cubic spline problem with the boundary conditions  $S''(x_0) = S''(x_n) = 0$  can be obtained by applying Algorithm 3.4.

#### ALGORITHM

### 3.4

#### Natural Cubic Spline

To construct the cubic spline interpolant  $S$  for the function  $f$ , defined at the numbers  $x_0 < x_1 < \dots < x_n$ , satisfying  $S''(x_0) = S''(x_n) = 0$ :

**INPUT**  $n; x_0, x_1, \dots, x_n; a_0 = f(x_0), a_1 = f(x_1), \dots, a_n = f(x_n)$ .

**OUTPUT**  $a_j, b_j, c_j, d_j$  for  $j = 0, 1, \dots, n-1$ .

(Note:  $S(x) = S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3$  for  $x_j \leq x \leq x_{j+1}$ .)

**Step 1** For  $i = 0, 1, \dots, n-1$  set  $h_i = x_{i+1} - x_i$ .

**Step 2** For  $i = 1, 2, \dots, n-1$  set

$$\alpha_i = \frac{3}{h_i}(a_{i+1} - a_i) - \frac{3}{h_{i-1}}(a_i - a_{i-1}).$$

**Step 3** Set  $l_0 = 1$ ; (Steps 3, 4, 5, and part of Step 6 solve a tridiagonal linear system using a method described in Algorithm 6.7.)

$$\mu_0 = 0;$$

$$z_0 = 0.$$

**Step 4** For  $i = 1, 2, \dots, n-1$

$$\text{set } l_i = 2(x_{i+1} - x_{i-1}) - h_{i-1}\mu_{i-1};$$

$$\mu_i = h_i/l_i;$$

$$z_i = (\alpha_i - h_{i-1}z_{i-1})/l_i.$$

**Step 5** Set  $l_n = 1$ ;

$$z_n = 0;$$

$$c_n = 0.$$

**Step 6** For  $j = n-1, n-2, \dots, 0$

$$\text{set } c_j = z_j - \mu_j c_{j+1};$$

$$b_j = (a_{j+1} - a_j)/h_j - h_j(c_{j+1} + 2c_j)/3;$$

$$d_j = (c_{j+1} - c_j)/(3h_j).$$

**Step 7** OUTPUT  $(a_j, b_j, c_j, d_j$  for  $j = 0, 1, \dots, n-1$ );  
STOP. ■

A result similar to Theorem 3.11 holds in the case of clamped boundary conditions.

**Theorem 3.12**

If  $f$  is defined at  $a = x_0 < x_1 < \dots < x_n = b$  and differentiable at  $a$  and  $b$ , then  $f$  has a unique clamped spline interpolant  $S$  on the nodes  $x_0, x_1, \dots, x_n$ ; that is, a spline interpolant that satisfies the boundary conditions  $S'(a) = f'(a)$  and  $S'(b) = f'(b)$ . ■

**Proof** Since  $f'(a) = S'(a) = S'(x_0) = b_0$ , Eq. (3.20) with  $j = 0$  implies

$$f'(a) = \frac{1}{h_0}(a_1 - a_0) - \frac{h_0}{3}(2c_0 + c_1).$$

Consequently,

$$2h_0c_0 + h_0c_1 = \frac{3}{h_0}(a_1 - a_0) - 3f'(a).$$

Similarly,

$$f'(b) = b_n = b_{n-1} + h_{n-1}(c_{n-1} + c_n),$$

so Eq. (3.20) with  $j = n-1$  implies that

$$\begin{aligned} f'(b) &= \frac{a_n - a_{n-1}}{h_{n-1}} - \frac{h_{n-1}}{3}(2c_{n-1} + c_n) + h_{n-1}(c_{n-1} + c_n) \\ &= \frac{a_n - a_{n-1}}{h_{n-1}} + \frac{h_{n-1}}{3}(c_{n-1} + 2c_n), \end{aligned}$$

and

$$h_{n-1}c_{n-1} + 2h_{n-1}c_n = 3f'(b) - \frac{3}{h_{n-1}}(a_n - a_{n-1}).$$

Equations (3.21) together with the equations

$$2h_0c_0 + h_0c_1 = \frac{3}{h_0}(a_1 - a_0) - 3f'(a)$$

and

$$h_{n-1}c_{n-1} + 2h_{n-1}c_n = 3f'(b) - \frac{3}{h_{n-1}}(a_n - a_{n-1})$$

determine the linear system  $Ax = b$ , where

$$A = \begin{bmatrix} 2h_0 & h_0 & 0 & \cdots & 0 \\ h_0 & 2(h_0 + h_1) & h_1 & \cdots & 0 \\ 0 & h_1 & 2(h_1 + h_2) & h_2 & \cdots \\ \vdots & \vdots & \vdots & \vdots & \ddots \\ 0 & \cdots & 0 & h_{n-1} & 2h_{n-1} \end{bmatrix},$$

$$b = \begin{bmatrix} \frac{3}{h_0}(a_1 - a_0) - 3f'(a) \\ \frac{3}{h_1}(a_2 - a_1) - \frac{3}{h_0}(a_1 - a_0) \\ \vdots \\ \frac{3}{h_{n-1}}(a_n - a_{n-1}) - \frac{3}{h_{n-2}}(a_{n-1} - a_{n-2}) \\ 3f'(b) - \frac{3}{h_{n-1}}(a_n - a_{n-1}) \end{bmatrix} \quad \text{and} \quad x = \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix}.$$

The matrix  $A$  is strictly diagonally dominant, so it satisfies the conditions of Theorem 6.19. Therefore, the linear system has a unique solution for  $c_0, c_1, \dots, c_n$ . ■ ■ ■

The solution to the cubic spline problem with the boundary conditions  $S'(x_0) = f'(x_0)$  and  $S'(x_n) = f'(x_n)$  can be obtained by applying Algorithm 3.5.

## ALGORITHM

## 3.5

**Clamped Cubic Spline**

To construct the cubic spline interpolant  $S$  for the function  $f$  defined at the numbers  $x_0 < x_1 < \cdots < x_n$ , satisfying  $S'(x_0) = f'(x_0)$  and  $S'(x_n) = f'(x_n)$ :

**INPUT**  $n; x_0, x_1, \dots, x_n; a_0 = f(x_0), a_1 = f(x_1), \dots, a_n = f(x_n); FPO = f'(x_0); FPN = f'(x_n)$ .

**OUTPUT**  $a_j, b_j, c_j, d_j$  for  $j = 0, 1, \dots, n-1$ .

(Note:  $S(x) = S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3$  for  $x_j \leq x \leq x_{j+1}$ .)

**Step 1** For  $i = 0, 1, \dots, n-1$  set  $h_i = x_{i+1} - x_i$ .

**Step 2** Set  $\alpha_0 = 3(a_1 - a_0)/h_0 - 3FPO$ ;  
 $\alpha_n = 3FPN - 3(a_n - a_{n-1})/h_{n-1}$ .

**Step 3** For  $i = 1, 2, \dots, n-1$

$$\text{set } \alpha_i = \frac{3}{h_i}(a_{i+1} - a_i) - \frac{3}{h_{i-1}}(a_i - a_{i-1}).$$

**Step 4** Set  $l_0 = 2h_0$ ; (Steps 4, 5, 6, and part of Step 7 solve a tridiagonal linear system using a method described in Algorithm 6.7.)

$$\mu_0 = 0.5;$$

$$z_0 = \alpha_0/l_0.$$



**Step 5** For  $i = 1, 2, \dots, n-1$   
     set  $l_i = 2(x_{i+1} - x_{i-1}) - h_{i-1}\mu_{i-1}$ ;  
      $\mu_i = h_i/l_i$ ;  
      $z_i = (\alpha_i - h_{i-1}z_{i-1})/l_i$ .

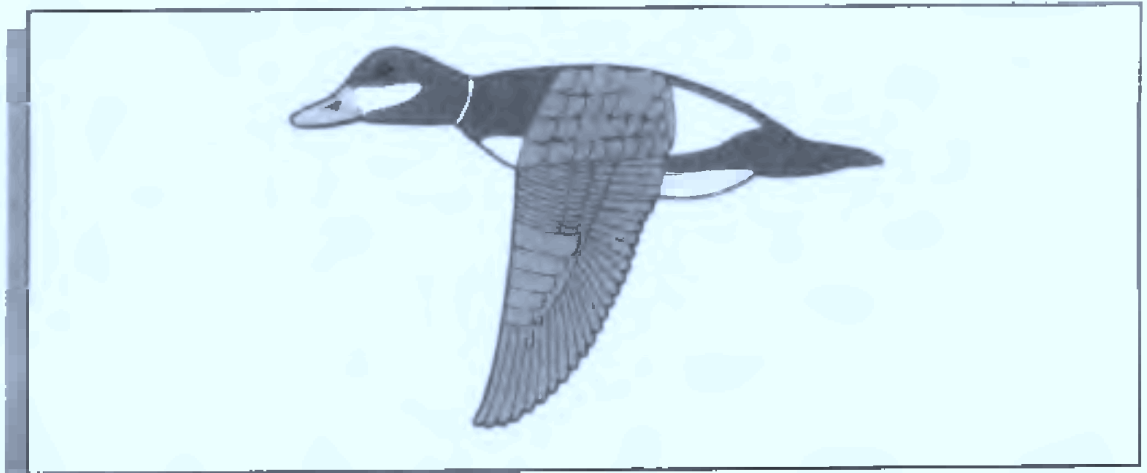
**Step 6** Set  $l_n = h_{n-1}(2 - \mu_{n-1})$ ;  
      $z_n = (\alpha_n - h_{n-1}z_{n-1})/l_n$ ;  
      $c_n = z_n$ .

**Step 7** For  $j = n-1, n-2, \dots, 0$   
     set  $c_j = z_j - \mu_j c_{j+1}$ ;  
      $b_j = (a_{j+1} - a_j)/h_j - h_j(c_{j+1} + 2c_j)/3$ ;  
      $d_j = (c_{j+1} - c_j)/(3h_j)$ .

**Step 8** OUTPUT  $(a_j, b_j, c_j, d_j$  for  $j = 0, 1, \dots, n-1)$ ;  
     STOP. ■

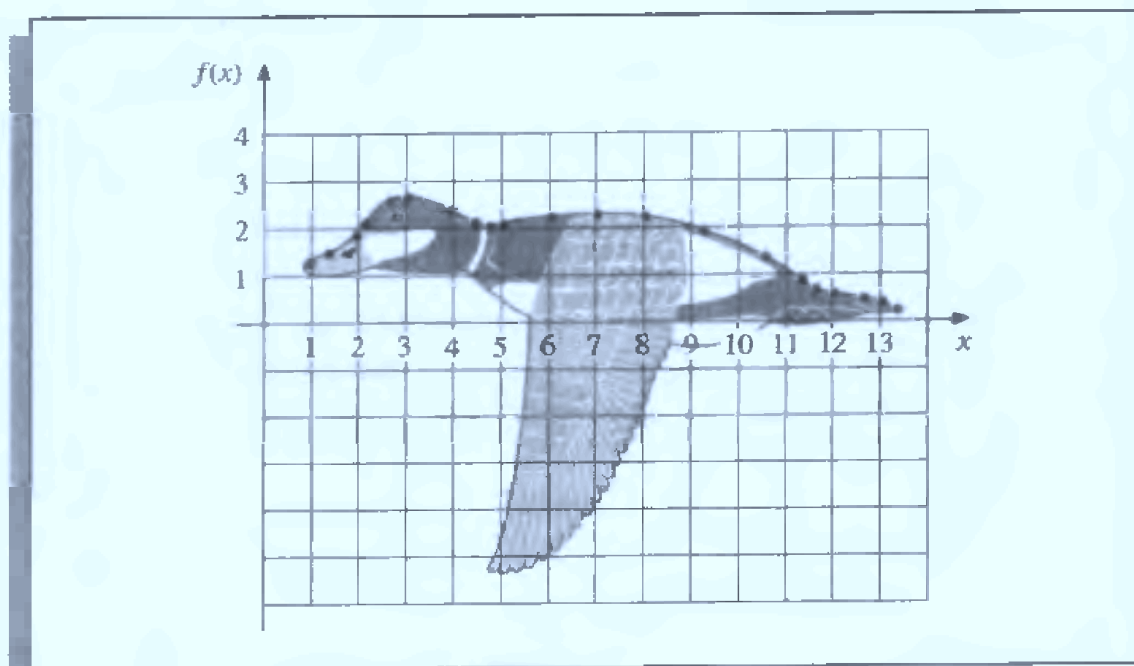
**EXAMPLE 1**

Figure 3.9 shows a ruddy duck in flight. To approximate the top profile of the duck, we have chosen points along the curve through which we want the approximating curve to pass. Table 3.15 lists the coordinates of 21 data points relative to the superimposed coordinate system shown in Figure 3.10 on page 150. Notice that more points are used when the curve is changing rapidly than when it is changing more slowly.

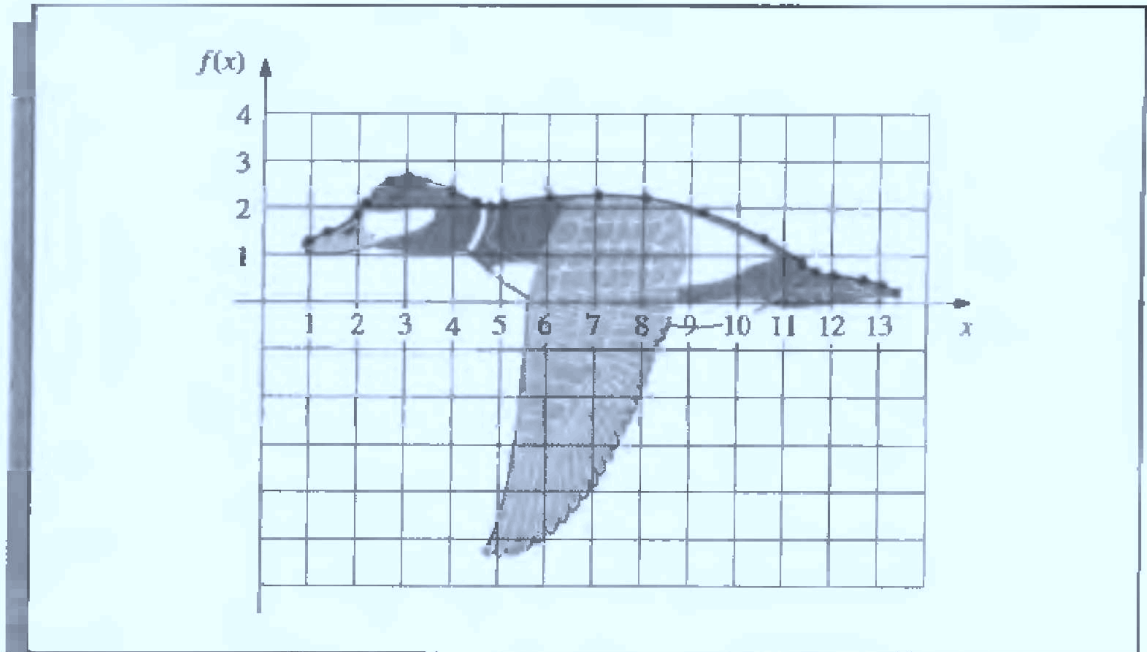
**Figure 3.9****Table 3.15**

| $x$    | 0.9 | 1.3 | 1.9  | 2.1 | 2.6 | 3.0 | 3.9 | 4.4  | 4.7  | 5.0 | 6.0  | 7.0 | 8.0  | 9.2  | 10.5 | 11.3 | 11.6 | 12.0 | 12.6 | 13.0 | 13.3 |
|--------|-----|-----|------|-----|-----|-----|-----|------|------|-----|------|-----|------|------|------|------|------|------|------|------|------|
| $f(x)$ | 1.3 | 1.5 | 1.85 | 2.1 | 2.6 | 2.7 | 2.4 | 2.15 | 2.05 | 2.1 | 2.25 | 2.3 | 2.25 | 1.95 | 1.4  | 0.9  | 0.7  | 0.6  | 0.5  | 0.4  | 0.25 |

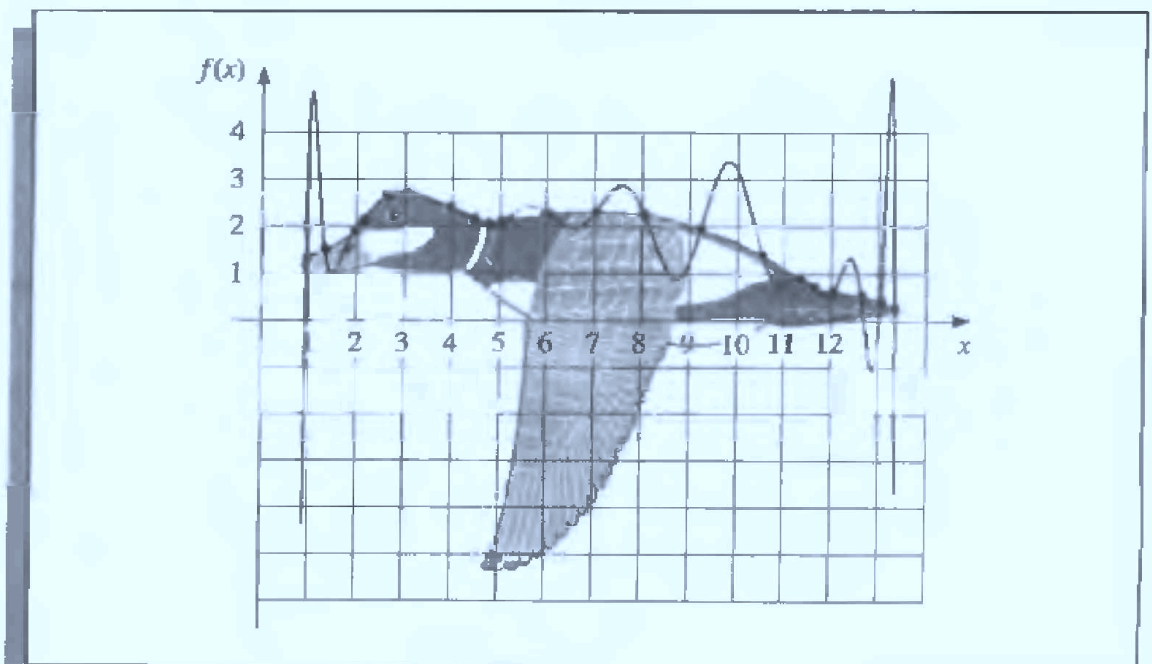
Using Algorithm 3.4 to generate the free cubic spline for this data produces the coefficients shown in Table 3.16. This spline curve is nearly identical to the profile, as shown in Figure 3.11.

**Figure 3.10****Table 3.16**

| $j$ | $x_j$ | $a_j$ | $b_j$ | $c_j$ | $d_j$ |
|-----|-------|-------|-------|-------|-------|
| 0   | 0.9   | 1.3   | 5.40  | 0.00  | -0.25 |
| 1   | 1.3   | 1.5   | 0.42  | -0.30 | 0.95  |
| 2   | 1.9   | 1.85  | 1.09  | 1.41  | -2.96 |
| 3   | 2.1   | 2.1   | 1.29  | -0.37 | -0.45 |
| 4   | 2.6   | 2.6   | 0.59  | -1.04 | 0.45  |
| 5   | 3.0   | 2.7   | -0.02 | -0.50 | 0.17  |
| 6   | 3.9   | 2.4   | -0.50 | -0.03 | 0.08  |
| 7   | 4.4   | 2.15  | -0.48 | 0.08  | 1.31  |
| 8   | 4.7   | 2.05  | -0.07 | 1.27  | -1.58 |
| 9   | 5.0   | 2.1   | 0.26  | -0.16 | 0.04  |
| 10  | 6.0   | 2.25  | 0.08  | -0.03 | 0.00  |
| 11  | 7.0   | 2.3   | 0.01  | -0.04 | -0.02 |
| 12  | 8.0   | 2.25  | -0.14 | -0.11 | 0.02  |
| 13  | 9.2   | 1.95  | -0.34 | -0.05 | -0.01 |
| 14  | 10.5  | 1.4   | -0.53 | -0.10 | -0.02 |
| 15  | 11.3  | 0.9   | -0.73 | -0.15 | 1.21  |
| 16  | 11.6  | 0.7   | -0.49 | 0.94  | -0.84 |
| 17  | 12.0  | 0.6   | -0.14 | -0.06 | 0.04  |
| 18  | 12.6  | 0.5   | -0.18 | 0.00  | -0.45 |
| 19  | 13.0  | 0.4   | -0.39 | -0.54 | 0.60  |
| 20  | 13.3  | 0.25  |       |       |       |

**Figure 3.11**

For comparison purposes, Figure 3.12 gives an illustration of the curve that is generated using a Lagrange interpolating polynomial to fit the data given in Table 3.15. This produces a very strange illustration of the back of a duck, in flight or otherwise. The interpolating polynomial in this case is of degree 20 and oscillates wildly.

**Figure 3.12**

To use a clamped spline to approximate this curve we would need derivative approximations for the endpoints. Even if these approximations were available, we could expect little improvement because of the close agreement of the free cubic spline to the curve of the top profile. ■

Constructing a cubic spline to approximate the lower profile of the ruddy duck would be more difficult since the curve for this portion cannot be expressed as a function of  $x$ , and at certain points the curve does not appear to be smooth. These problems can be resolved by using separate splines to represent various portions of the curve, but a more effective approach to curves of this type is considered in the next section.

The clamped boundary conditions are generally preferred when approximating functions by cubic splines, so the derivative of the function must be estimated at the endpoints of the interval. When the nodes are equally spaced near both endpoints, approximations can be obtained by using Eq. (4.7) or any of the other appropriate formulas given in Sections 4.1 and 4.2. When the nodes are unequally spaced, the problem is considerably more difficult.

To conclude this section, we list an error-bound formula for the cubic spline with clamped boundary conditions. The proof of this result can be found in [Schul, pp. 57–58].

**Theorem 3.13**

Let  $f \in C^4[a, b]$  with  $\max_{a \leq x \leq b} |f^{(4)}(x)| = M$ . If  $S$  is the unique clamped cubic spline interpolant to  $f$  with respect to the nodes  $a = x_0 < x_1 < \cdots < x_n = b$ , then

$$\max_{a \leq x \leq b} |f(x) - S(x)| \leq \frac{5M}{384} \max_{0 \leq j \leq n-1} (x_{j+1} - x_j)^4. \quad \blacksquare$$

A fourth-order error-bound result also holds in the case of free boundary conditions, but it is more difficult to express. (See [BD, pp. 827–835].)

The free boundary conditions will generally give less accurate results than the clamped conditions near the ends of the interval  $[x_0, x_n]$  unless the function  $f$  happens to nearly satisfy  $f''(x_0) = f''(x_n) = 0$ . An alternative to the free boundary condition that does not require knowledge of the derivative of  $f$  is the *not-a-knot* condition, (see [Deb, pp. 55–56]). This condition requires that  $S'''(x)$  be continuous at  $x_1$  and at  $x_{n-1}$ .

## EXERCISE SET 3.4

- Determine the free cubic spline  $S$  that interpolates the data  $f(0) = 0$ ,  $f(1) = 1$ , and  $f(2) = 2$ .
- Determine the clamped cubic spline  $s$  that interpolates the data  $f(0) = 0$ ,  $f(1) = 1$ ,  $f(2) = 2$  and satisfies  $s'(0) = s'(2) = 1$ .
- Construct the free cubic spline for the following data.

a.

| $x$ | $f(x)$   |
|-----|----------|
| 8.3 | 17.56492 |
| 8.6 | 18.50515 |

b.

| $x$ | $f(x)$     |
|-----|------------|
| 0.8 | 0.22363362 |
| 1.0 | 0.65809197 |

| c. | $x$   | $f(x)$     |
|----|-------|------------|
|    | -0.5  | -0.0247500 |
|    | -0.25 | 0.3349375  |
|    | 0     | 1.1010000  |

| d. | $x$ | $f(x)$      |
|----|-----|-------------|
|    | 0.1 | -0.62049958 |
|    | 0.2 | -0.28398668 |
|    | 0.3 | 0.00660095  |
|    | 0.4 | 0.24842440  |

4. The data in Exercise 3 were generated using the following functions. Use the cubic splines constructed in Exercise 3 for the given value of  $x$  to approximate  $f(x)$  and  $f'(x)$ , and calculate the actual error.
- $f(x) = x \ln x$ ; approximate  $f(8.4)$  and  $f'(8.4)$ .
  - $f(x) = \sin(e^x - 2)$ ; approximate  $f(0.9)$  and  $f'(0.9)$ .
  - $f(x) = x^3 + 4.001x^2 + 4.002x + 1.101$ ; approximate  $f(-\frac{1}{3})$  and  $f'(-\frac{1}{3})$ .
  - $f(x) = x \cos x - 2x^2 + 3x - 1$ ; approximate  $f(0.25)$  and  $f'(0.25)$ .
5. Construct the clamped cubic spline using the data of Exercise 3 and the fact that
- $f'(8.3) = 1.116256$  and  $f'(8.6) = 1.151762$
  - $f'(0.8) = 2.1691753$  and  $f'(1.0) = 2.0466965$
  - $f'(-0.5) = 0.7510000$  and  $f'(0) = 4.0020000$
  - $f'(0.1) = 3.58502082$  and  $f'(0.4) = 2.16529366$
6. Repeat Exercise 4 using the cubic splines constructed in Exercise 5.
7. A natural cubic spline  $S$  on  $[0, 2]$  is defined by

$$S(x) = \begin{cases} S_0(x) = 1 + 2x - x^3, & \text{if } 0 \leq x < 1, \\ S_1(x) = 2 + b(x-1) + c(x-1)^2 + d(x-1)^3, & \text{if } 1 \leq x \leq 2. \end{cases}$$

Find  $b$ ,  $c$ , and  $d$ .

8. A clamped cubic spline  $s$  for a function  $f$  is defined on  $[1, 3]$  by

$$s(x) = \begin{cases} s_0(x) = 3(x-1) + 2(x-1)^2 - (x-1)^3, & \text{if } 1 \leq x < 2, \\ s_1(x) = a + b(x-2) + c(x-2)^2 + d(x-2)^3, & \text{if } 2 \leq x \leq 3. \end{cases}$$

Given  $f'(1) = f'(3)$ , find  $a$ ,  $b$ ,  $c$ , and  $d$ .

9. A natural cubic spline  $S$  is defined by

$$S(x) = \begin{cases} S_0(x) = 1 + B(x-1) - D(x-1)^3, & \text{if } 1 \leq x < 2, \\ S_1(x) = 1 + b(x-2) - \frac{3}{4}(x-2)^2 + d(x-2)^3, & \text{if } 2 \leq x \leq 3. \end{cases}$$

If  $S$  interpolates the data  $(1, 1)$ ,  $(2, 1)$ , and  $(3, 0)$ , find  $B$ ,  $D$ ,  $b$ , and  $d$ .

10. A clamped cubic spline  $s$  for a function  $f$  is defined by

$$s(x) = \begin{cases} s_0(x) = 1 + Bx + 2x^2 - 2x^3, & \text{if } 0 \leq x < 1, \\ s_1(x) = 1 + b(x-1) - 4(x-1)^2 + 7(x-1)^3, & \text{if } 1 \leq x \leq 2. \end{cases}$$

Find  $f'(0)$  and  $f'(2)$ .

11. Construct a free cubic spline to approximate  $f(x) = \cos \pi x$  by using the values given by  $f(x)$  at  $x = 0, 0.25, 0.5, 0.75$ , and  $1.0$ . Integrate the spline over  $[0, 1]$ , and compare the result to  $\int_0^1 \cos \pi x \, dx = 0$ . Use the derivatives of the spline to approximate  $f'(0.5)$  and  $f''(0.5)$ . Compare these approximations to the actual values.
12. Construct a free cubic spline to approximate  $f(x) = e^{-x}$  by using the values given by  $f(x)$  at  $x = 0, 0.25, 0.75$ , and  $1.0$ . Integrate the spline over  $[0, 1]$ , and compare the result to

$\int_0^1 e^{-x} dx = 1 - 1/e$ . Use the derivatives of the spline to approximate  $f'(0.5)$  and  $f''(0.5)$ . Compare the approximations to the actual values.

13. Repeat Exercise 11, constructing instead the clamped cubic spline with  $f'(0) = f'(1) = 0$ .
14. Repeat Exercise 12, constructing instead the clamped cubic spline with  $f'(0) = -1$ ,  $f'(1) = -e^{-1}$ .
15. Suppose that  $f(x)$  is a polynomial of degree 3. Show that  $f(x)$  is its own clamped cubic spline, but that it cannot be its own free cubic spline.
16. Suppose the data  $\{x_i, f(x_i)\}_{i=1}^n$  lie on a straight line. What can be said about the free and clamped cubic splines for the function  $f$ ? [Hint: Take a cue from the results of Exercises 1 and 2.]
17. Given the partition  $x_0 = 0$ ,  $x_1 = 0.05$ , and  $x_2 = 0.1$  of  $[0, 0.1]$ , find the piecewise linear interpolating function  $F$  for  $f(x) = e^{2x}$ . Approximate  $\int_0^{0.1} e^{2x} dx$  with  $\int_0^{0.1} F(x) dx$ , and compare the results to the actual value.
18. Let  $f \in C^2[a, b]$ , and let the nodes  $a = x_0 < x_1 < \cdots < x_n = b$  be given. Derive an error estimate similar to that in Theorem 3.13 for the piecewise linear interpolating function  $F$ . Use this estimate to derive error bounds for Exercise 17.
19. Extend Algorithms 3.4 and 3.5 to include as output the first and second derivatives of the spline at the nodes.
20. Extend Algorithms 3.4 and 3.5 to include as output the integral of the spline over the interval  $[x_0, x_n]$ .
21. Given the partition  $x_0 = 0$ ,  $x_1 = 0.05$ ,  $x_2 = 0.1$  of  $[0, 0.1]$  and  $f(x) = e^{2x}$ :
  - a. Find the cubic spline  $s$  with clamped boundary conditions that interpolates  $f$ .
  - b. Find an approximation for  $\int_0^{0.1} e^{2x} dx$  by evaluating  $\int_0^{0.1} s(x) dx$ .
  - c. Use Theorem 3.13 to estimate  $\max_{0 \leq x \leq 0.1} |f(x) - s(x)|$  and

$$\left| \int_0^{0.1} f(x) dx - \int_0^{0.1} s(x) dx \right|.$$

- d. Determine the cubic spline  $S$  with free boundary conditions, and compare  $S(0.02)$ ,  $s(0.02)$ , and  $e^{0.04} = 1.04081077$ .
22. Let  $f$  be defined on  $[a, b]$ , and let the nodes  $a = x_0 < x_1 < x_2 = b$  be given. A quadratic spline interpolating function  $S$  consists of the quadratic polynomial

$$S_0(x) = a_0 + b_0(x - x_0) + c_0(x - x_0)^2 \quad \text{on } [x_0, x_1]$$

and the quadratic polynomial

$$S_1(x) = a_1 + b_1(x - x_1) + c_1(x - x_1)^2 \quad \text{on } [x_1, x_2],$$

such that

- i.  $S(x_0) = f(x_0)$ ,  $S(x_1) = f(x_1)$ , and  $S(x_2) = f(x_2)$ ,
- ii.  $S \in C^1[x_0, x_2]$ .

Show that conditions (i) and (ii) lead to five equations in the six unknowns  $a_0$ ,  $b_0$ ,  $c_0$ ,  $a_1$ ,  $b_1$ , and  $c_1$ . The problem is to decide what additional condition to impose to make the solution unique. Does the condition  $S \in C^2[x_0, x_2]$  lead to a meaningful solution?

23. Determine a quadratic spline  $s$  that interpolates the data  $f(0) = 0$ ,  $f(1) = 1$ ,  $f(2) = 2$  and satisfies  $s'(0) = 2$ .

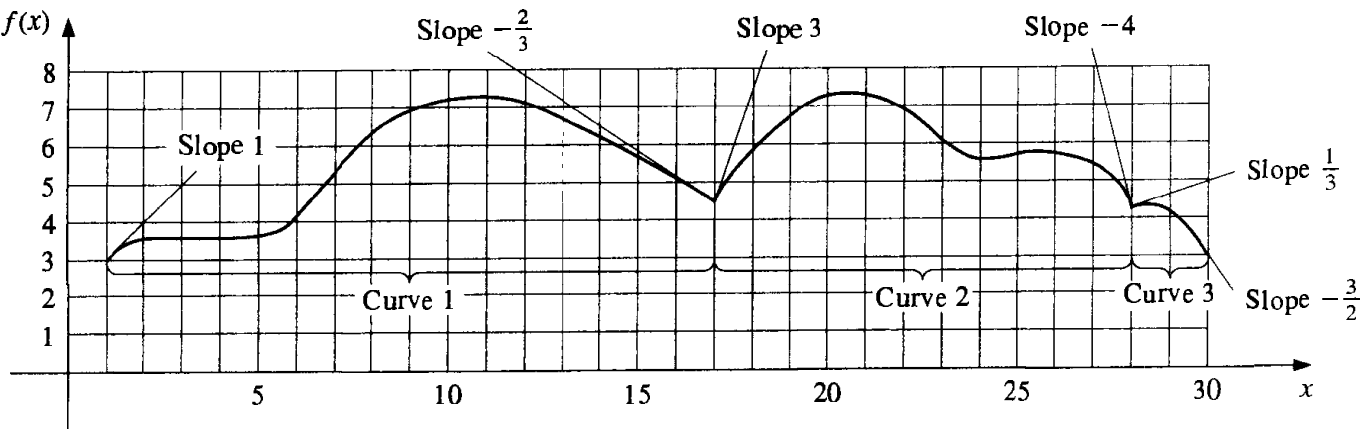
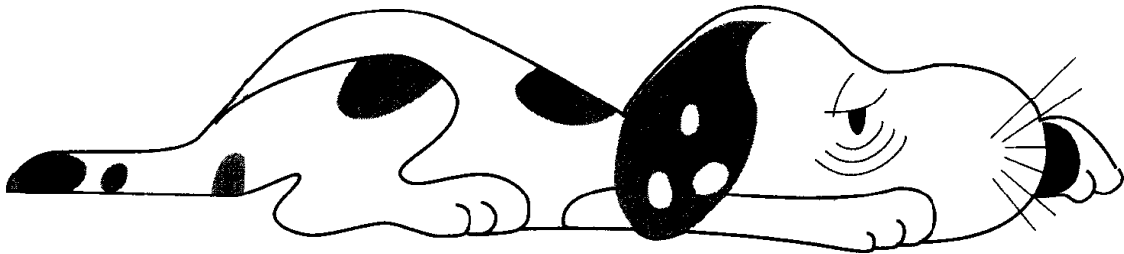
24. a. The introduction to this chapter included a table listing the population of the United States from 1940 to 1990. Use free cubic spline interpolation to approximate the population in the years 1930, 1965, and 2010.
- b. The population in 1930 was approximately 123,203,000. How accurate do you think your 1965 and 2010 figures are?
25. A car traveling along a straight road is clocked at a number of points. The data from the observations are given in the following table, where the time is in seconds, the distance is in feet, and the speed is in feet per second.

|          |    |     |     |     |     |
|----------|----|-----|-----|-----|-----|
| Time     | 0  | 3   | 5   | 8   | 13  |
| Distance | 0  | 225 | 383 | 623 | 993 |
| Speed    | 75 | 77  | 80  | 74  | 72  |

- a. Use a clamped cubic spline to predict the position of the car and its speed when  $t = 10$  s.
- b. Use the derivative of the spline to determine whether the car ever exceeds a 55-mi/h speed limit on the road; if so, what is the first time the car exceeds this speed?
- c. What is the predicted maximum speed for the car?
26. The 1995 Kentucky Derby was won by a horse named Thunder Gulch in a time of  $2:01\frac{1}{5}$  (2 min and  $\frac{1}{5}$  s) for the  $1\frac{1}{4}$ -mi race. Times at the quarter-mile, half-mile, and mile poles were  $22\frac{2}{5}$ ,  $45\frac{4}{5}$ , and  $1:35\frac{3}{5}$ .
- a. Use these values together with the starting time to construct a free cubic spline for Thunder Gulch's race.
- b. Use the spline to predict the time at the three-quarter-mile pole, and compare this to the actual time of  $1:10\frac{1}{5}$ .
- c. Use the spline to approximate Thunder Gulch's starting speed and speed at the finish line.
27. It is suspected that the high amounts of tannin in mature oak leaves inhibit the growth of the winter moth (*Operophtera bromata* L., *Geometridae*) larvae that extensively damage these trees in certain years. The following table lists the average weight of two samples of larvae at times in the first 28 days after birth. The first sample was reared on young oak leaves, whereas the second sample was reared on mature leaves from the same tree.
- a. Use a free cubic spline to approximate the average weight curve for each sample.
- b. Find an approximate maximum average weight for each sample by determining the maximum of the spline.

|                              |      |       |       |       |       |       |       |
|------------------------------|------|-------|-------|-------|-------|-------|-------|
| Day                          | 0    | 6     | 10    | 13    | 17    | 20    | 28    |
| Sample 1 average weight (mg) | 6.67 | 17.33 | 42.67 | 37.33 | 30.10 | 29.31 | 28.74 |
| Sample 2 average weight (mg) | 6.67 | 16.11 | 18.89 | 15.00 | 10.56 | 9.44  | 8.89  |

28. The upper portion of this noble beast is to be approximated using clamped cubic spline interpolants. The curve is drawn on a grid from which the table is constructed. Use Algorithm 3.5 to construct the three clamped cubic splines.



| Curve 1 |       |          |           | Curve 2 |       |          |           | Curve 3 |       |          |           |
|---------|-------|----------|-----------|---------|-------|----------|-----------|---------|-------|----------|-----------|
| $i$     | $x_i$ | $f(x_i)$ | $f'(x_i)$ | $i$     | $x_i$ | $f(x_i)$ | $f'(x_i)$ | $i$     | $x_i$ | $f(x_i)$ | $f'(x_i)$ |
| 0       | 1     | 3.0      | 1.0       | 0       | 17    | 4.5      | 3.0       | 0       | 27.7  | 4.1      | 0.33      |
| 1       | 2     | 3.7      |           | 1       | 20    | 7.0      |           | 1       | 28    | 4.3      |           |
| 2       | 5     | 3.9      |           | 2       | 23    | 6.1      |           | 2       | 29    | 4.1      |           |
| 3       | 6     | 4.2      |           | 3       | 24    | 5.6      |           | 3       | 30    | 3.0      | -1.5      |
| 4       | 7     | 5.7      |           | 4       | 25    | 5.8      |           |         |       |          |           |
| 5       | 8     | 6.6      |           | 5       | 27    | 5.2      |           |         |       |          |           |
| 6       | 10    | 7.1      |           | 6       | 27.7  | 4.1      | -4.0      |         |       |          |           |
| 7       | 13    | 6.7      |           |         |       |          |           |         |       |          |           |
| 8       | 17    | 4.5      | -0.67     |         |       |          |           |         |       |          |           |

29. Repeat Exercise 28, constructing three natural splines using Algorithm 3.4.

### 3.5 Parametric Curves

None of the techniques developed in this chapter can be used to generate curves of the form shown in Figure 3.13 since this curve cannot be expressed as a function of one coordinate variable in terms of the other. In this section we will see how to represent general curves by using a parameter to express both the  $x$ - and  $y$ -coordinate variables. This technique can be extended to represent general curves and surfaces in space.